

Course Code: AE8137

Course Title: Advanced Systems Control

Semester: F2021

Instructor: Prof. Guangjun Liu

Project

Title: Fundamental Yaw Control System for a Horizontal Axis Wind Turbine

Section Number:

Submission Date: 11/19/2021

Due Date: 11/23/2021

Name	Student Number (XXXX99999)	Signature
Godswill Ezeorah	501012886	G.E.

By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: www.ryerson.ca/senate/current/pol60.pdf.

Abstract

The design of an active yaw control system for a horizontal axis wind turbine (HAWT) is quite complex and challenging, especially for mega turbines. This paper tries to give a simple and fundamental approach to designing a suitable control system, using some modern control theorem. To archive this some assumptions were considered, such as a fixed rotor azimuth angle, blade pitch angle and that the rotor velocity is a known constant. These assumptions help reduce the problem from multiple degrees of freedom (DOF) problem to a 1-DOF (i.e., just the yaw motion) problem. Furthermore, to make the mathematical modelling simple and comprehensible, Euler's method of dynamics formulation is employed to derive the equation of motion using the sum of torque (to maintain equilibrium). And the control system was designed in state-space form, using the feedback linearization method and then the control law feedback gains were estimated using the pole placement method. The new closed-loop model was programmed in Simulink software. And the result simulation process was automated within MATLAB, for seamless analysis of the plotted results. And a resulting stable and moderate yaw control system was obtained for tracking the wind direction of the modelled system.

Table of Contents

1.	Introduction.....	4
2.	Mathematical Model	4
3.	Control System Design	7
4.	Simulation and Result	10
5.	Conclusion	13
6.	References.....	14
	APPENDIX A.....	15

List of Tables

Table 1 Desired Tracking Trajectory	8
Table 2 Wind Turbine Physical Parameters.....	10

List of Figures

Figure 1 Wind Turbine Axis Parameter.....	5
Figure 2 Close Loop Control System Schematics (Simulink).....	10
Figure 3 Case1 [tracking response (left) and Torque input(right)]	11
Figure 4 Case2 [tracking response (left) and Torque input(right)].....	11
Figure 5 Case3 [tracking response (left) and Torque input(right)].....	12

1. Introduction

Having a yaw system in a horizontal turbine is very important. Because the efficiency of the turbine can be increased if the rotor is always redirected to face the incoming wind, which could change direction at random (unlike the sun). This means that the controller has to be responsive and stable. The formulation of the equation of motion for the turbine can be very complicated as shown in the reference article [1]. Hence the objective of the project is to develop the fundamental control system for the yaw control of a horizontal axis wind turbine.

Consequently, the mathematical modelling of the yaw dynamics plant will be done in matrix form, instead of indexed notation. As it is easier for researchers and engineers to understand the fundamentals and utilize this paper for any future application. And based on the assumptions made to reduce the problem to a 1-DOF problem, i.e., just the yaw dynamics, and friction and any breaking systems will not be considered in the modelling. Thus this model is expected to be a non-linear dynamic for the plant. Hence feedback linearization will be the best approach in designing a stable control system for the model.

Finally, the close loop state-space linearized model will be programmed with the MATLAB Simulink package because the control system block diagram is illustrative within the Simulink software. Also, the simulation to be carried out is easy to visualize within MATLAB, as to observe the behaviour of the system for different control performance specifications.

2. Mathematical Model

Using Euler's equation of motion, we have the sum of angular momentum as,

$$\vec{H} = \vec{H}_{yaw} + \vec{h}_{rotor}$$

And resolving the components of the rotor angular momentum vector to the x-axis and y-axis of the reference frame. Assuming that the rotor axis is perpendicular to the yaw axis and thus has no z component.

$$\vec{h}_{rotor} = -h_r(\hat{i} \cos \phi + \hat{j} \sin \phi)$$

$$h_r = I_{ry}\dot{\theta}$$

Where, I_{ry} is the moment of inertial about the rotor y-axis centroid, and $\dot{\theta}$ is the angular velocity of the rotor, measure from the rotor body frame.

$$\vec{H}_{yaw} = I_z\dot{\phi}\hat{k}$$

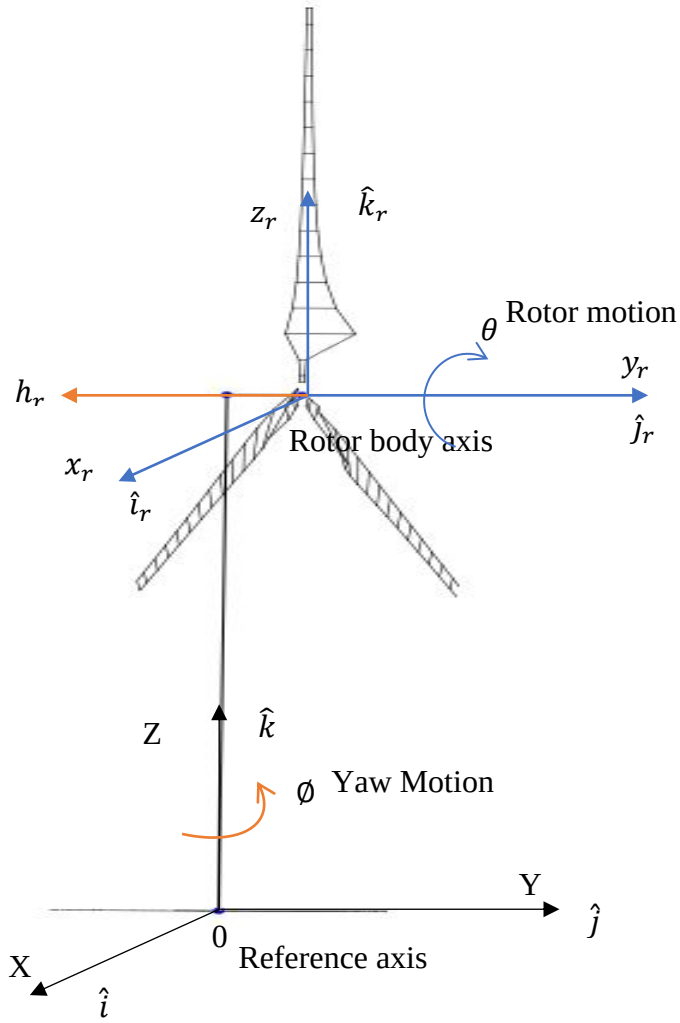


Figure 1 Wind Turbine Axis Parameter

Where, I_z is the moment of inertial about the yaw Z-axis centroid, and $\dot{\phi}$ is the angular velocity of the yaw motion, measure from the reference frame.

So that the overall angular momentum acting on the system is,

$$\vec{H} = \begin{bmatrix} I_x \omega_x \\ I_y \omega_y \\ I_z \omega_z \end{bmatrix} = \begin{bmatrix} -I_{ry} \dot{\theta} \cos \phi \\ -I_{ry} \dot{\theta} \sin \phi \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ I_z \dot{\phi} \end{bmatrix}$$

$$\vec{H} = \begin{bmatrix} -I_{ry} \dot{\theta} \cos \phi \\ -I_{ry} \dot{\theta} \sin \phi \\ I_z \dot{\phi} \end{bmatrix} \quad (1.1)$$

This gives the angular velocity vector as,

$$\vec{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} \frac{-I_{ry}\dot{\theta} \cos \phi}{I_x} \\ \frac{-I_{ry}\dot{\theta} \sin \phi}{I_y} \\ \dot{\phi} \end{bmatrix} \quad (1.2)$$

The torque for the above system will be calculated as,

$$\vec{T} = \frac{d\vec{H}}{dt} + \vec{\omega} \times \vec{H}$$

Where the skew matrix $\vec{\omega} \times$ can be rewritten as, $\vec{\omega} \times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$

Differentiating equation (1.1) with respect to time and applying the chain rule (i.e., phi and theta are time-dependent). The torque will now be written as,

$$\begin{aligned} \vec{T} &= \begin{bmatrix} -I_{ry}\ddot{\theta} \cos \phi + I_{ry}\dot{\theta}\dot{\phi} \sin \phi \\ -I_{ry}\ddot{\theta} \sin \phi - I_{ry}\dot{\theta}\dot{\phi} \cos \phi \\ I_z\ddot{\phi} \end{bmatrix} + \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \begin{bmatrix} -I_{ry}\dot{\theta} \cos \phi \\ -I_{ry}\dot{\theta} \sin \phi \\ I_z\dot{\phi} \end{bmatrix} \\ \vec{T} &= \begin{bmatrix} -I_{ry}\ddot{\theta} \cos \phi + I_{ry}\dot{\theta}\dot{\phi} \sin \phi \\ -I_{ry}\ddot{\theta} \sin \phi - I_{ry}\dot{\theta}\dot{\phi} \cos \phi \\ I_z\ddot{\phi} \end{bmatrix} + \begin{bmatrix} \omega_z I_{ry}\dot{\theta} \sin \phi + \omega_y I_z\dot{\phi} \\ -\omega_z I_{ry}\dot{\theta} \sin \phi - \omega_x I_z\dot{\phi} \\ \omega_y I_{ry}\dot{\theta} \cos \phi - \omega_x I_{ry}\dot{\theta} \sin \phi \end{bmatrix} \end{aligned}$$

Substituting equation (1.1) into the above, and performing algebraic and trigonometric simplification (i.e. $\cos \phi \sin \phi = \frac{\sin 2\phi}{2}$). The equation of motion becomes,

$$\vec{T} = \begin{bmatrix} T_x \\ T_y \\ T_z \end{bmatrix} = \begin{bmatrix} -I_{ry}\ddot{\theta} \cos \phi + 2I_{ry}\dot{\theta}\dot{\phi} \sin \phi - \frac{I_z I_{ry}\dot{\theta}\dot{\phi} \sin \phi}{I_y} \\ -I_{ry}\ddot{\theta} \sin \phi - 2I_{ry}\dot{\theta}\dot{\phi} \cos \phi + \frac{I_z I_{ry}\dot{\theta}\dot{\phi} \sin \phi}{I_x} \\ I_z\ddot{\phi} - \frac{I_{ry}^2 \dot{\theta}^2 \sin 2\phi}{2I_y} + \frac{I_{ry}^2 \dot{\theta}^2 \sin 2\phi}{2I_x} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ T_z \end{bmatrix}$$

The assumption is made that rotor speed ($\dot{\theta}$) is a known constant so that dropping the torque equation about the x-axis and y-axis, will leave a 1-D torque equation. The re-arrangement and simplification of this gives,

$$\ddot{\phi} = \frac{T_z}{I_z} + \frac{K_{moi} I_{ry}^2 \dot{\theta}^2 \sin 2\phi}{2}$$

Where, $K_{moi} = \frac{I_x - I_y}{I_x I_y I_z}$

And this is the yaw dynamics (plant), that will be utilized for the control system design in state space. Although the rotor angular velocity ($\dot{\theta}$) has been assumed to be a constant, but in practice, it should vary with the wind speed.

3. Control System Design

The control system design is done using the feedback linearization method since the plant model is in the companion form. And its state-space form is given as;

$$\begin{aligned}\dot{x}_1 &= x_2 \\ \dot{x}_2 &= f(x_1) + Bu\end{aligned}\quad (2.0)$$

Where, $x_1 = \emptyset$, and the non-linear term, $f(x_1) = \frac{K_{moi} I_{ry}^2 \dot{\theta}^2 \sin 2x_1}{2}$, the scalar control input, $u = T_z$, with, $B = \frac{1}{I_z}$

And to cancel the non-linear terms, let the control input be,

$$u = \frac{1}{B}(v - f(x_1))$$

And substituting this into the state space equation, yields,

$$\ddot{x}_1 = \dot{x}_2 = v \quad (2.1)$$

Which is in the form of a simple linear equation, so that using the linear feedback control method, the control law becomes,

$$v = -k_1 x_1 - k_2 x_2$$

But this project aims to be able to track the direction of the wind for efficient power generation. Hence the new control law can be written as,

$$v = \ddot{x}_{1d} - k_1 e - k_2 \dot{e} \quad (2.2)$$

Where, the error terms, $e = (x_1 - x_{1d})$ and $\dot{e} = (\dot{x}_1 - \dot{x}_{1d})$

For our case study, the desired input is the angular position of the wind direction, assuming that we don't have a continuous derivative function of this position with respect to time, as the wind direction has a stochastic behaviour. We can assume that the initial reading from the sensor (wind vane) for the wind direction was 30 *deg*, and it's subsequent assumed noiseless/accurately measured wind direction at any instance of time, for a sample of 10sec reading are:

Table 1 Desired Tracking Trajectory

Timestep	Desired position, x_{1d} [deg]	Desired velocity, $\dot{x}_{1d}$ [deg/sec]	Desired Acceleration, $\ddot{x}_{1d}$ [deg/sec ²]
1	30	0	0
2	30	0	0
3	30	0	0
4	50	20	20
5	70	20	0
6	90	20	0
7	70	-20	-40
8	50	-20	0
9	30	-20	0
10	30	0	20

This data set will be imported into the Simulink software as the desired tracking input for the simulation. Observe that there will be a sharp change in the plot of the desired position with time. This is done to simulate the control system behaviour at such a rapid wind direction change scenario.

Now integrating the above control law into the original non-linear system, and the new control input will be,

$$u = T_z = I_z(\ddot{x}_{1d} - k_1\dot{e} - k_2\ddot{e} - f(x_1))$$

So that simply substituting into equation (2.1)) and having it in terms of x_1 will give the new closed-loop dynamics for the control system, that is **Exponentially Stable** (i.e., tracking error, $e \rightarrow 0$ at any time instance, or exponentially convergent tracking),

$$\ddot{e} + k_2\dot{e} + k_1e = 0 \quad (2.3)$$

N/B: the control law is valid for all state variables, except if the moment of inertial, $I_z = \infty$ (singularity), which is less likely to happen, hence the system is said to be globally stable). The above equation leads to a linear behaviour of the original dynamics, with the assumption that the model and computer calculations are 100% accurate, with no disturbance, which is not true in the real-world scenario.

Taking the Laplace transform of the above equation, the characteristic equation will be,

$$s^2 + k_2s + k_1 = 0 \quad (2.4)$$

So that using the **pole placement** method the feedback gain (k_1 and k_2) can be determined for a given performance criterion, such as percentage maximum overshoot (%OS), settling time (T_s), peak time (T_p), etc, thus the damping ratio ζ , natural frequency ω_n the parameter can be obtained. And such desired characteristic equation can be written as,

$$s^2 + 2 \zeta \omega_n s + \omega_n^2 = 0 \quad (2.5)$$

Therefore, choosing to use %OS and T_s (i.e., within $\pm 2\%$ from the steady-state value) as the performance specification for this paper, the damping ratio ζ , can be estimated as,

$$\zeta = \frac{-\ln\left(\% \frac{OS}{100}\right)}{\sqrt{\pi^2 + \ln^2\left(\% \frac{OS}{100}\right)}}$$

And after that natural frequency ω_n , is obtained with,

$$\omega_n = \frac{4}{T_s \zeta}$$

Directly comparing both equations (2.4) and (2.5), gives the feedback gain as,

$$k_1 = \frac{16}{T_s^2} \left(\frac{\pi^2}{\ln^2\left(\% \frac{OS}{100}\right)} + 1 \right) \quad (2.6)$$

$$k_2 = \frac{8}{T_s} \quad (2.7)$$

With the above, the feedback gain values can be automatically solved using the MATLAB code provided in Appendix A. With this, we can re-write the exponentially stable error dynamics from equation (2.0) as,

$$\dot{x}_2 = \frac{K_{moi} I_{ry}^2 \dot{\theta}^2 \sin 2x_1}{2} + \frac{1}{I_z} (I_z (\ddot{x}_{1d} - k_1 e - k_2 \dot{e} - f(x_1)))$$

And this is the close loop system as represented below (in Simulink). Although the non-linear terms in the above equation will be cancelled, so that the system behaves like a linear system (i.e., as in equation (2.3)), but the new torque term is also important for the simulation analysis.

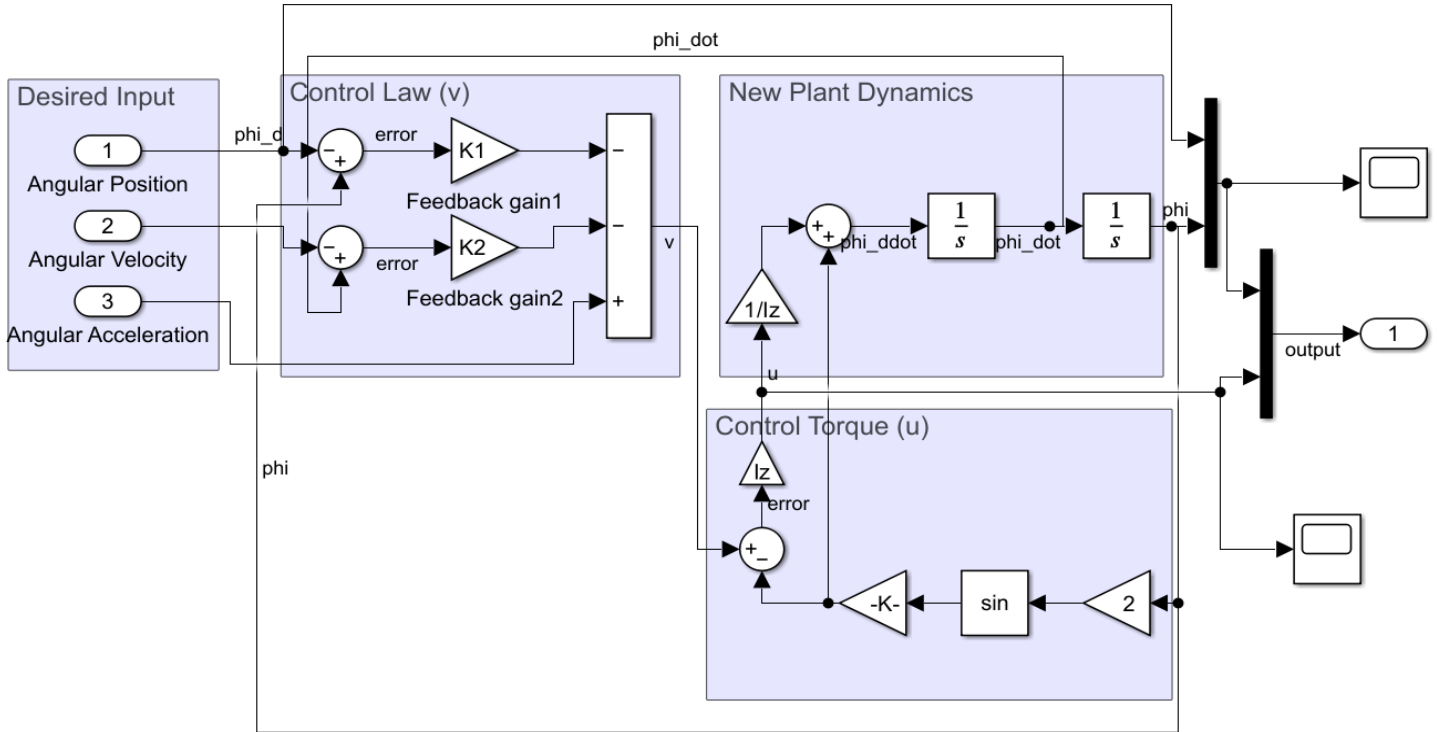


Figure 2 Close Loop Control System Schematics (Simulink)

4. Simulation and Result

For the simulation, the following three performance cases were considered:

Case1 (High performance requirement): $\%OS = 0\%$ and $T_s = 0.012\text{ s}$

Case2 (Low performance requirement): $\%OS = 0\%$ and $T_s = 0.34133\text{ s}$

Case3 (Moderate performance requirement): $\%OS = 8\%$ and $T_s = 0.34133\text{ s}$

The simulation is not restricted to just these three cases, and with the well commented MATLAB code provided in Appendix A. With this many ranges of performance specifications to be simulated, the author automated the simulation process directly from MATLAB and hence making the simulation and result evaluation hassle-free from manually updating variables and running Simulink one after the other for each performance case.

And the physical parameters for the model are:

Table 2 Wind Turbine Physical Parameters

Parameters	Values
Moment of inertial (due to nacelle, hub and rotor), $[I_x, I_y, I_z]$	$[1.2, 0.5, 0.9]\text{ kg.m}^2$
Moment of inertial (due to rotor), I_{ry}	0.07 kg.m^2
Rotor angular velocity (for 3m blade radius), $\dot{\theta}$	9 m/s

In the Simulink model solver settings, all settings are left in their default settings, except the **relative tolerance**, which is adequately set to $1e^{-12}$, although this will increase the computation time for convergence, it will also improve the accuracy (about $1e^{-10}\%$) of the model state solution as shown below:

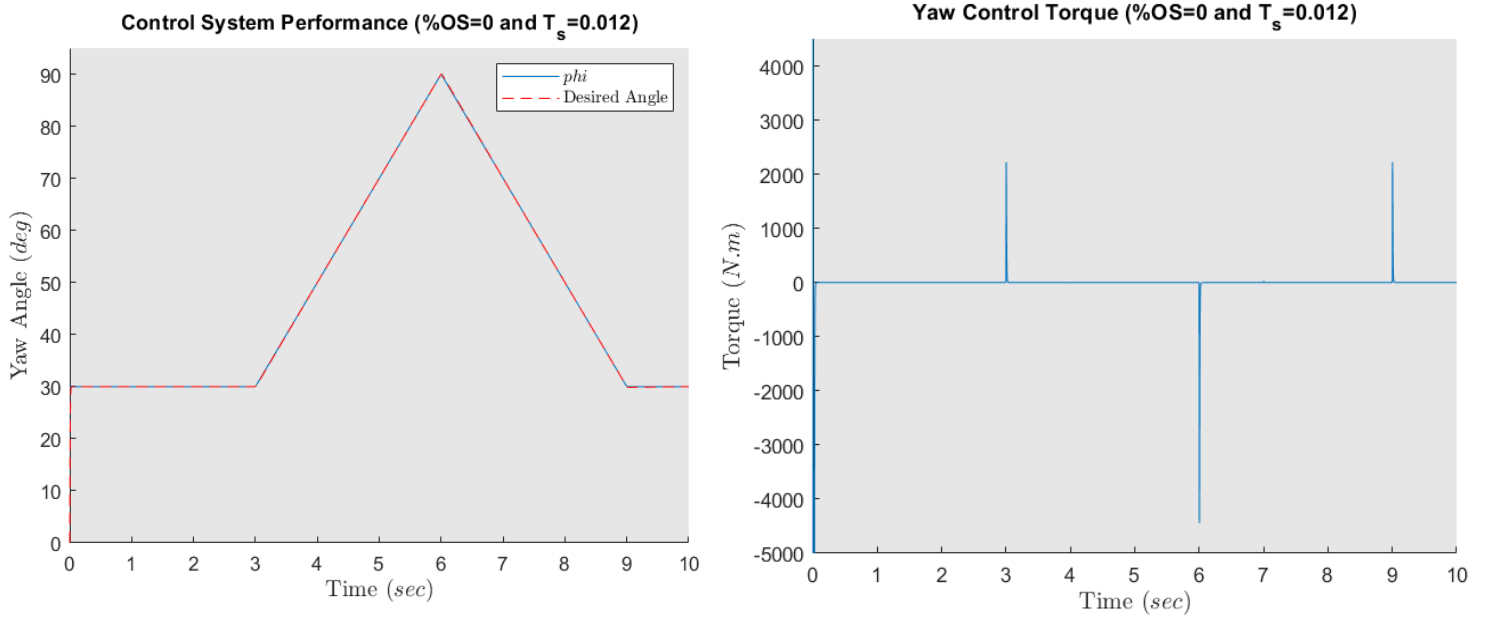


Figure 3 Case1 [tracking response (left) and Torque input(right)]

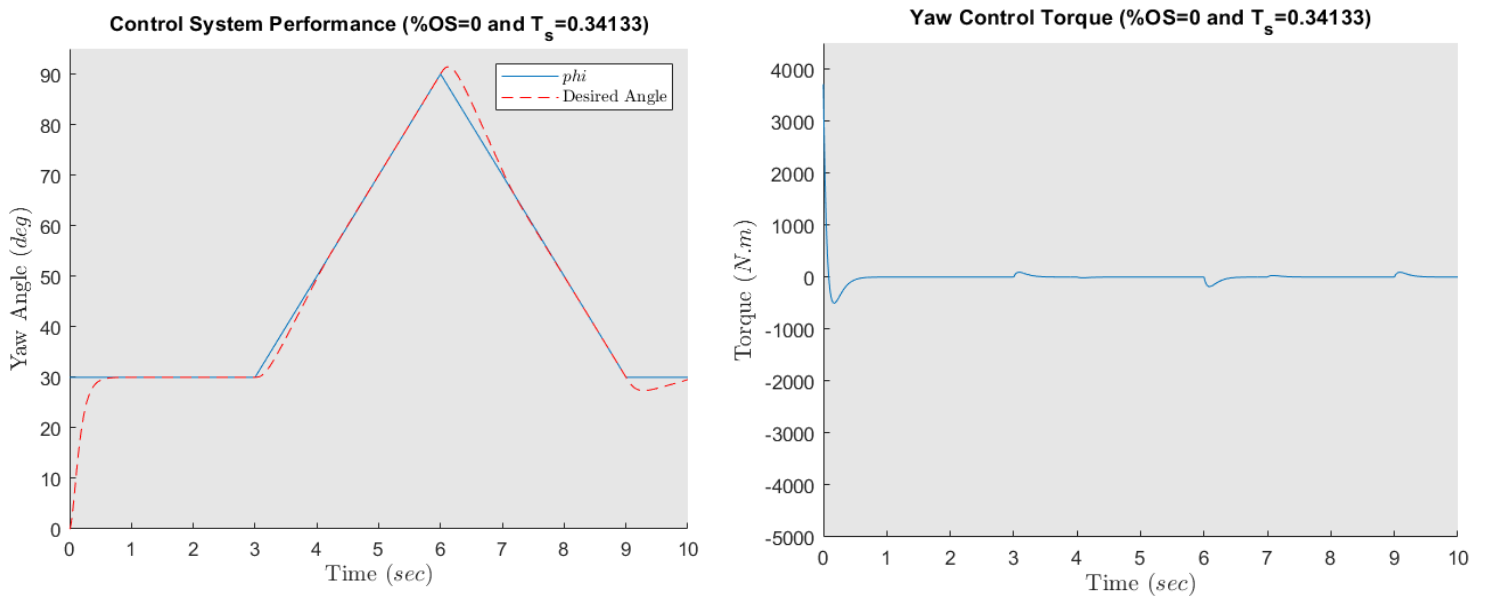


Figure 4 Case2 [tracking response (left) and Torque input(right)]

From the above result, notice that for case1, the tracking response is fairly smooth at the sharp wind direction change, whereas case2, overshoots a little at about (2 – 8.7)% around the three sharp-edge regions. So that just increasing the percentage overshoot requirement as in case3, the

underdamped tracking behaviour at the sharp-edges in case2 can be reduced to about (0.5 – 2)% overshoot from the steady-state for the three regions of steep wind direction change, as shown below:

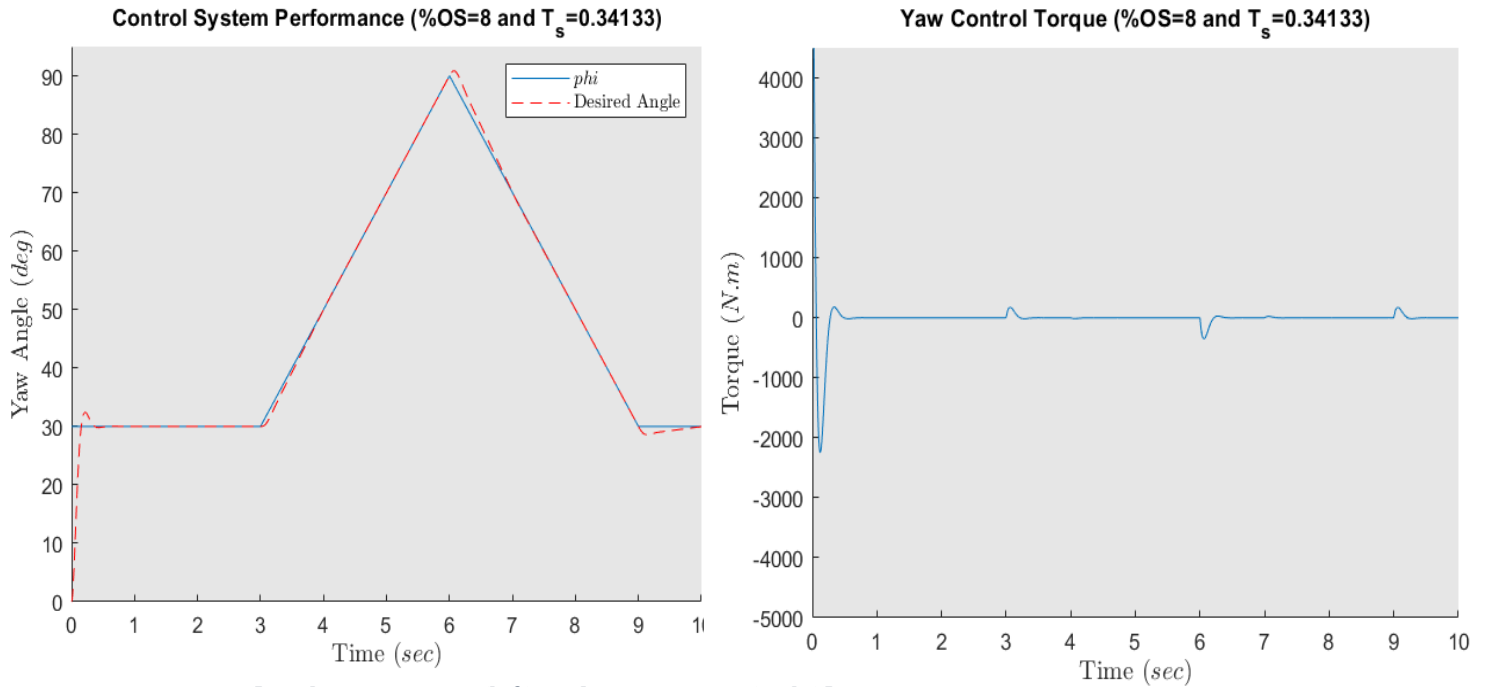


Figure 5 Case3 [tracking response (left) and Torque input(right)]

Intuitively, the anticipated torque input for the three sharp-edge regions shows spick/rapid responses (high torque at a very small time interval) for high-performance case1. Practically speaking getting a motor that can deliver such high torques and response, will be very difficult. And in practice, mega wind turbines, which require active yaw control use large electric motors coupled with planetary gear which makes the system have higher torque and at a reduced response. But the torque level for case2 and case3 is more practical. Also, observe that increasing the overshoot from case2 to case3 (at constant settling time), increases the torque input slightly, at almost the same response time interval. This explains why the underdamped behaviour at the three regions where slightly minimized. And thus yielding a moderate tracking response for the yaw control system.

Zooming in on the plot, it is observed that for case3 the overshoot value is precisely at 32.4 deg, (i.e., at 8% overshoot) which suggests that the feedback gain values were accurately computed. Also, note that the steady-state torque reading is not zero, but at an average of about 0.014 N.m when zoomed-in on the torque plot, with negligible variation for each case study.

5. Conclusion

After the formulation of the mathematical model, for the non-linear yaw dynamics, the control system design was carried out using state feedback linearization. And a new close-loop linear dynamics were obtained, with feedback gain values, which are estimated using the pole placement method for the desired performance specification (i.e., percentage overshoot and settling time). The simulation process was automated with the MATLAB code (Appendix A) by integrating Simulink into the code. The following conclusion is made from the simulated results:

- A stable control system was obtained for a solver of relative tolerance equal to $1e^{12}$.
- The high-performance specification (as in case3) is practically difficult to achieve with current motors and their accompanying noise factor. And thus suggesting a more moderate performance requirement for practical application (as in case2), with a percentage overshoot of 8% instead of 0%.

For future work, the friction, motor (with or without gearbox) and braking system model can be incorporated in the model, for creating a model that is more close to the real system. Also, a robust control strategy can be considered for future control design, since the rotor angular velocity is a variable and not a constant (as assumed in this paper), which acts as a payload by exacting torque (through the rotor angular momentum) on the yaw dynamics.

6. References

- [1] E. Campos-Mercado, L. F. Cerecero-Natale, O. Garcia-Salazar, H. F. Abundis Fong and D. Wood, "Mathematical modeling and fuzzy proportional–integral–derivative scheme to control the yaw motion of a wind turbine," *Wind Energy*, vol. 24, no. 4, pp. 379-401, 4 2021.
- [2] J. Dai, T. He, M. Li and X. Long, "Performance study of multi-source driving yaw system for aiding yaw control of wind turbines," *Renewable Energy*, vol. 163, pp. 154-171, 1 2021.
- [3] J. Yang, L. Fang, D. Song, M. Su, X. Yang, L. Huang and Y. H. Joo, "Review of control strategy of large horizontal-axis wind turbines yaw system," *Wind Energy*, vol. 24, no. 2, pp. 97-115, 2 2021.
- [4] F. A. Khan and S. Nisar, "Design and analysis of feedback control system," pp. 16-24, 5 2018.
- [5] K. Ogata, *Modern control engineering*, Prentice-Hall, 2010, p. 894.
- [6] J.-J. E. (.-J. E. Slotine and W. Li, *Applied nonlinear control*, Prentice Hall, 1991, p. 459.

APPENDIX A

```
% -----  
% Written by Godswill Ezeorah for AE8137 Course Project on  
% 13-Nov-2021, to automate Simulink Simulation  
% -----  
clear; clc;  
%% Variable Initialization [table (2)]  
Ix = 1.2; Iy = 0.5; Iz = 0.9;           % Refrence axis Moment of inertial  
Iry = 0.07;                           % Rotor y-axis Moment of inertial  
theta_dot = 9;                         % Rotor angular velocity for raduis=3m)  
Kmoi = (Ix-Iy)/Ix*Iy*Iz;  
R_tol = 1e-12;                         % Solver Reletive Tolerance  
%% Desired Inputs for Tracking  
wind = [30;30;30;50;70;90;70;50;30;30]; % Angular position  
wind_dot = [0;0;0;20;20;20;-20;-20;-20;0]; % Angular velocity  
wind_ddot = [0;0;0;20;0;0;-40;0;0;20]; % Angular accleration  
time_step = [1;2;3;4;5;6;7;8;9;10]; % corresponding time  
  
%% Control Performance Specification [table (1)]  
Tsmax = 1; Tsmmin = 0.012;  
OS = [0,0,8];                         % range Percentage Overshot  
[r,n]=size(OS);  
mT = (Tsmax-Tsmmin)/n;  
T_s = (Tsmmin:mT:Tsmax);               % range Settling Time  
[row,col] = size(OS);  
  
%% Looping Through all Performance Specification in Simulink  
for i=1 :col  
    % Feedback gain definition  
    K1 = (16/T_s(i)^2)*(pi^2/log(OS(i)/100)^2+1); % Equation (2.6)  
    K2 = 8/T_s(i); % Equation (2.7)  
  
    %% Extraction of Simulink Output Dataset  
    sim1=sim('yaw_control.slx'); %Gets the simulink simulation dataset  
    sig = sim1.yout.getElement('output'); %Extracts time structured output signal  
    t = sig.Values.time; %Extracts time array  
    phi_a = sig.Values.Data(:,2); %Extracts measured position  
    phi_d = sig.Values.Data(:,1); %Extracts desired position  
    torque = sig.Values.Data(:,3); %Extracts the new torque signal  
  
    %% Plotting the Simulation Results  
    fig1 = figure(i); ax1 = axes; hold(ax1,'on'); % for Control plot  
    plot(ax1,t, phi_d);  
    plot(ax1,t, phi_a,'-r');  
    leg1 = legend('$\phi$', 'Desired Angle');  
    set(leg1, 'Interpreter', 'latex');  
    title(['Control System Performance (%OS=', num2str(OS(i)), ...
```



```

    ' and T_s=',num2str(T_s(i)),')]);
xlabel('Time $(sec)$','Interpreter','latex','fontsize',12);
ylabel('Yaw Angle $(deg)$','Interpreter','latex','fontsize',12);
axis([0 10 0 95]);
set(gca,'Color',[0.9,0.9,0.9]);

fig2 = figure(i+3); ax2 = axes; hold(ax2,'on'); % for Torque plot
plot(ax2,t, torque);
title(['Yaw Control Torque (%OS=',num2str(OS(i)),...
    ' and T_s=',num2str(T_s(i)),')']);
xlabel('Time $(sec)$','Interpreter','latex','fontsize',12);
ylabel('Torque $(N.m)$','Interpreter','latex','fontsize',12);
axis([0 10 -5e3 4.5e3]);
set(gca,'Color',[0.9,0.9,0.9]);
end
1

```